

Sparse Vector Range Reverse

(1 sec, 64mb)

CP::sparse_vector เป็นโครงสร้างข้อมูลแบบหนึ่ง ซึ่งมีการใช้ในลักษณะเดียวกับ CP::vector กล่าวคือเราสามารถอ้างถึงช่องต่าง ๆ ได้โดยใช้หมายเลข 0, 1, 2, ... อย่างไรก็ตามในการใช้งาน CP::sparse_vector นั้น จะมีแค่บางช่องที่มีข้อมูล โดยช่องส่วนใหญ่จะถือว่าไม่มีข้อมูลอยู่ภายใน ตัวอย่างเช่น เราอาจจะมี CP::sparse_vector ที่เก็บข้อมูลประเภท int ที่มีการเก็บข้อมูลดังนี้

ช่องที่	0	1	2	3	4	5	6	7
ค่า	4	-	-	3	-	4	7	1

การเก็บข้อมูลนั้นจะใช้ std::map ในการเก็บ โดยเก็บเฉพาะช่องที่มีข้อมูล ช่องที่ไม่มีข้อมูลจะไม่ถูกแสดงออกมา

โดยในโจทย์ข้อนี้เราต้องการจะทำการ reverse ช่วง ๆ หนึ่งของ CP::sparse_vector อันนี้โดยเมื่อเรียกใช้ฟังก์ชัน void CP::sparse_vector:: range_reserve (int l, int r) ซึ่งจะทำการรับขอบซ้าย (l) และขอบขวา (r) เข้ามา จากนั้น reverse ช่วงนั้นๆ เช่นหากใช้ CP::sparse_vector อันเดิมและเรียกใช้ range_reserve (2, 7) จะได้ผลลัพธ์ดังนี้

ช่องที่	0	1	2	3	4	5	6	7
ค่า	4	-	1	7	4	-	3	-

สังเกตว่าถึงแม้บางช่องจะไม่มีค่าแต่ก็จำเป็นต้องเป็นส่วนหนึ่งที่โดน reverse ไปด้วย ในอีกตัวอย่างหนึ่งเช่นหากมี CP::sparse_vector ที่มีค่าเป็น

ช่องที่	0	1	2	3	4	5	6	7
ค่า	-	3	-	-	2	-	-	100

หากมีการเรียก range_reserve (1, 4) จะได้ CP::sparse_vector ใหม่มีค่าดังนี้

ช่องที่	0	1	2	3	4	5	6	7
ค่า	-	2	-	-	3	-	-	100

ข้อบังคับ

- โจทย์ข้อนี้จะมีไฟล์ตั้งต้นมาให้ ซึ่งประกอบด้วยไฟล์ `sparse_vector.h`, `main.cpp` และ `student.h` อยู่ ให้นักศึกษาเขียน code เพิ่มเติมลงในไฟล์ `student.h` เท่านั้น และการส่งไฟล์เข้าสู่ระบบ grader ให้ส่งเฉพาะไฟล์ `student.h` เท่านั้น
 - ไฟล์ `student.h` จะต้องไม่ทำการอ่านเขียนข้อมูลใด ๆ ไปยังหน้าจอหรือคีย์บอร์ดหรือไฟล์ใด ๆ
 - หากใช้ VS Code ให้ทำการ compile ที่ไฟล์ `main.cpp`
- ** main ที่ใช้จริงใน grader นั้นจะแตกต่างจาก main**

ที่ได้รับในไฟล์โปรเจกต์เริ่มต้นแต่จะทำการทดสอบในลักษณะเดียวกัน **

คำอธิบายฟังก์ชัน main

`main()` จะเป็นการทดลองใช้งาน sparse vector โดย main จะอ่านข้อมูลนำเข้าดังนี้

- บรรทัดแรก เป็นจำนวนเต็ม `T` ระบุจำนวนรอบของการทดสอบสำหรับแต่ละรอบการทดสอบ

จากนั้น main ทำการสร้าง `CP::sparse_vector<int> sv` ขึ้นมา และจะรับข้อมูลนำเข้าแบบต่อเนื่องจนกว่าจะเจอคำสั่ง "q" โดยมีรูปแบบคำสั่งที่เป็นไปได้ทั้งหมดดังต่อไปนี้

- `sz` หมายถึงให้พิมพ์ค่าของ `sv.size()` ออกทางจอ
- `pb X` หมายถึงให้เรียก `sv.push_back(X)`
- `in X` หมายถึงให้เรียก `sv.insert(X)` โดยจะไม่มีกรเรียกใน main จริง
- `rz X` หมายถึงให้เรียก `sv.resize(X)`
- `rr l r` หมายถึงให้เรียก `sv.range_reverse(l, r)`
- `q` หมายถึงให้ main หยุดทำงาน

เมื่อออกจากการทำงานในแต่ละรอบของการทดสอบ โปรแกรมจะแสดง จำนวนข้อมูล, และข้อมูลทุกตัวใน `sparse_vector` ตามลำดับ

ชุดข้อมูลทดสอบ

ให้ `n` เป็นจำนวนครั้งที่มีการเรียก operation บวกกับจำนวนค่าที่อยู่ในช่วงที่มีการเรียก `range_reverse` จะมีการให้คะแนนดังนี้

- 20% $n \leq 100$
- 20% $n \leq 2,000$
- 60% $n \leq 500,000$

ตัวอย่างการทำงานของ main

ข้อมูลนำเข้า	ข้อมูลส่งออก
1 pb 4 rz 3 pb 3 rz 5 pb 4 pb 7 pb 1 rr 2 7 q	Sparse Vector Size: 8 Sparse Vector Content: [0] = 4 [2] = 1 [3] = 7 [4] = 4 [6] = 3
1 pb 5 pb 6 rz 6 rr 0 4 rz 8 rr 3 6 rz 40 pb 109 rz 100 pb 1 rz 321 rr 39 111 q	Sparse Vector Size: 321 Sparse Vector Content: [5] = 5 [6] = 6 [50] = 1 [110] = 109